

Informe:

Ejercicio 1:

Problema1: K número más pequeño

```
import heapq
import time

data_original = [3, 100, 20, 2, 80, 1]

data_heap = list(data_original)
heapq.heapify(data_heap)

inicio_heap = time.perf_counter()
ordenados_heap = []
while data_heap:
    ordenados_heap.append(heapq.heappop(data_heap))
fin_heap = time.perf_counter()

data_sorted = list(data_original)
inicio_sort = time.perf_counter()
ordenados_sort = sorted(data_sorted)
fin_sort = time.perf_counter()

print(f"--- RESULTADOS PROBLEMA 1 ---")
print(f"lista convertida en heap: {data_heap}")
print(f"lista convertida en heap (sorted): {data_sorted}")
print(f"lista ordenada de menor a mayor: {ordenados_heap}")
print(f"lista ordenada de menor a mayor: {ordenados_sort}")
print(f"Tiempo Heap (pop manual): {fin_heap - inicio_heap:.8f} seg")
print(f"Tiempo sorted(): {fin_sort - inicio_sort:.8f} seg")
```

```
--- RESULTADOS PROBLEMA 1 ---
lista convertida en heap: []
lista convertida en heap (sorted): [3, 100, 20, 2, 80, 1]
lista ordenada de menor a mayor: [1, 2, 3, 20, 80, 100]
lista ordenada de menor a mayor: [1, 2, 3, 20, 80, 100]
Tiempo Heap (pop manual): 0.00000480 seg
Tiempo sorted(): 0.00000180 seg
```

- Se uso heapify para transformar la lista data en un heap (min_heap), se imprime el heap y luego existe una función ordenados la cual ordena los números del heap de menor a mayor (podríamos decir que rompe la estructura del heap para ordenar los números de mayor a menor sin importar cual es raíz o hijo) .
- Se implementa una versión sorted del problema
- Comparativa de tiempo arroja que la versión sorted tarda menos que la versión implementada de forma manual.

Problema2: K elementos más frecuentes

```
import heapq
import time
from collections import Counter

data_original = [3, 100, 20, 2, 80, 1, 30, 20, 1, 3, 200, 3] * 1000

# TU IMPLEMENTACIÓN (Heap + Counter)
data_heap2 = list(data_original)
heapq.heapify(data_heap2)
conteos = Counter(data_heap2)

inicio_p2_heap = time.perf_counter()
repetidos = {num: veces for num, veces in conteos.items() if veces > 1}
ordenados_p2_heap = []
while data_heap2:
    ordenados_p2_heap.append(heapq.heappop(data_heap2))
fin_p2_heap = time.perf_counter()

# IMPLEMENTACIÓN LINEAL/SORTED
data_lin = list(data_original)
inicio_p2_lin = time.perf_counter()
# Encontrar repetidos de forma lineal
conteos_lin = Counter(data_lin)
repetidos_lin = {num: veces for num, veces in conteos_lin.items() if veces > 1}
# Ordenar
ordenados_p2_lin = sorted(data_lin)
fin_p2_lin = time.perf_counter()

print(f"\n--- RESULTADOS PROBLEMA 2 ---")
print(f"lista convertida en heap: {data_original}")
print(f"lista convertida en heap (sorted metod): {data_lin}")
print(f"repetidos y frecuencia {repetidos}")
print(f"repetidos y frecuencia (sort) {repetidos}")
print(f"Tiempo Heap: {fin_p2_heap - inicio_p2_heap:.8f} seg")
print(f"Tiempo Sort ed: {fin_p2_lin - inicio_p2_lin:.8f} seg")
```

[illegible]

- Se actualiza la lista del primer problema, se añade el método conteo para contar los elementos en data y ver que se repitan más de 1 vez y estos se imprimirán al final.
- Se implementa una versión sorted del problema.
- Muestra de comparativas de tiempo con el método manual y el sorted

Ejercicio 2

Problema 1:

```
import heapq
import time

def k_esimo_mas_grande(nums, k):
    inicio = time.perf_counter()
    min_heap = nums[:k]
    heapq.heapify(min_heap)

    for i in range(k, len(nums)):
        if nums[i] > min_heap[0]:
            heapq.heapreplace(min_heap, nums[i])

    resultado = min_heap[0]

    fin = time.perf_counter()

    return resultado, fin - inicio

# Datos de prueba
nums = [3, 2, 1, 5, 6, 4]
k = 2

res, duracion = k_esimo_mas_grande(nums, k)

print(f"Arreglo: {nums}")
print(f"El {k}-ésimo número más grande es: {res}")
print(f"Tiempo de ejecución: {duracion:.8f} segundos")
```

```
Arreglo: [3, 2, 1, 5, 6, 4]
El 2-ésimo número más grande es: 5
Tiempo de ejecución: 0.00000890 segundos

Process finished with exit code 0
```

- Se define el problema donde nums será nuestra lista y k el valor de orden que se busca
- Se crea el heap con los primeros K elementos
- Se recorren el resto de los números (ciclo for), si el número es mayor que el más pequeño del heap se reemplaza

Problema 2:

```
import heapq
import time

def mezclar_k_listas(listas): 1 usage new *

    inicio = time.perf_counter()

    min_heap = []
    resultado = []

    # Insertamos el primer elemento de cada lista en el heap
    # Guardamos una tupla: (valor, indice_de_la_lista, indice_del_elemento)
    for i in range(len(listas)):
        if listas[i]: # Verificamos que la lista no esté vacía
            heapq.heappush(*args: min_heap, (listas[i][0], i, 0))

    while min_heap:
        valor, lista_idx, elemento_idx = heapq.heappop(min_heap)
        resultado.append(valor)

        # Si la lista de la que salió el elemento tiene más números,
        # insertamos el siguiente en el heap
        if elemento_idx + 1 < len(listas[lista_idx]):
            siguiente_valor = listas[lista_idx][elemento_idx + 1]
            heapq.heappush(*args: min_heap, (siguiente_valor, lista_idx, elemento_idx + 1))

    fin = time.perf_counter()

    return resultado, fin - inicio

listas_entrada = [[1, 4, 5], [1, 3, 4], [2, 6]]

res, duracion = mezclar_k_listas(listas_entrada)

print(f"Entrada: {listas_entrada}")
print(f"Salida mezclada: {res}")
print(f"Tiempo de ejecución: {duracion:.8f} segundos")

Entrada: [[1, 4, 5], [1, 3, 4], [2, 6]]
Salida mezclada: [1, 1, 2, 3, 4, 4, 5, 6]
Tiempo de ejecución: 0.00001060 segundos
```

-Se establece el objetivo de unificar $\$k\$$ arreglos previamente ordenados en una sola estructura lineal, manteniendo la ordenación ascendente de manera eficiente.

- Se implementa la carga inicial del montículo insertando el primer elemento de cada lista. Cada entrada se almacena como una **tupla** (valor, índice_lista, índice_elemento). Esta estructura es vital para rastrear el origen de cada dato y saber de qué lista extraer el siguiente valor.

-Se incluye una validación para omitir listas vacías, evitando errores de índice durante la fase de inicialización

- Mediante un ciclo que aprovecha la propiedad del **Min-Heap**, se extrae sistemáticamente el valor mínimo global. Tras cada extracción, el algoritmo realiza una operación de "reemplazo" insertando el siguiente elemento de la lista de origen

Ejercicio 3:

```
import heapq
import time

def obtener_top_k_productos(productos, k):
    """usage new*"""
    inicio = time.perf_counter()
    min_heap = []
    for nombre, puntaje in productos:
        if len(min_heap) < k:
            heapq.heappush(*args: min_heap, (puntaje, nombre))
        else:
            if puntaje > min_heap[0][0]:
                heapq.heapreplace(min_heap, item: (puntaje, nombre))

    resultado = [nombre for puntaje, nombre in sorted(min_heap, reverse=True)]

    fin = time.perf_counter()

    return resultado, fin - inicio

# Datos de prueba
productos = [("Laptop", 95), ("Mouse", 80), ("Teclado", 85)]
k = 2

res, duracion = obtener_top_k_productos(productos, k)

print(f"Productos evaluados: {productos}")
print(f"Top-{k} productos: {res}")
print(f"Tiempo de ejecución: {duracion:.8f} segundos")

Productos evaluados: [('Laptop', 95), ('Mouse', 80), ('Teclado', 85)]
Top-2 productos: ['Laptop', 'Teclado']
Tiempo de ejecución: 0.00001260 segundos
```

- Identificar y extraer los k elementos con las valoraciones más altas dentro de un conjunto de datos estructurado (tuplas de producto y puntaje).
- Se implementa un montículo de tamaño fijo k. La estrategia consiste en comparar cada producto nuevo con la "base" del Top-K (el elemento con el puntaje más bajo dentro de los mejores). Si el nuevo producto supera este umbral, el elemento inferior es descartado y reemplazado.
- Para facilitar la comparación automática de heapq, se procesan los datos como (puntaje, nombre), permitiendo que el algoritmo ordene naturalmente por valor numérico.

